

Day 7: Hardware acceleration and inference runtimes

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 5, 2026

- ▶ Days 1 to 6 made the model smaller. From today the device is the Raspberry Pi 5: a Linux computer with four processor cores and 8 GB of RAM.
- ▶ On this device, the same model can run in more than one runtime. The runtime, its kernels, the number of threads, and the precision change the latency.
- ▶ A faster processor does not help a workload that waits for memory. The Roofline model shows which of the two limits applies.
- ▶ In the lab you run MobileNetV2 on the Raspberry Pi, export one model to ONNX and to LiteRT, and measure the latency for each runtime, thread count, and precision.

After this day you can:

- ▶ Explain how SIMD, accelerators, and memory bandwidth limit inference speed.
- ▶ Name the main graph optimizations and state what each one does.
- ▶ Export a PyTorch model to ONNX and to LiteRT.
- ▶ Compare inference runtimes on the Raspberry Pi.
- ▶ Explain the trade-off between static and dynamic input shapes.

Part 1 Acceleration fundamentals

Exercise: place three workloads on a Roofline plot

Part 2 The runtime stack

Exercise: select a runtime for three deployment cases

Part 3 Graph optimization

Exercise: fuse a convolution, a batch normalization, and a ReLU on paper

A new device, a new question

	XIAO ESP32S3	Raspberry Pi 5
Used on	Days 1 to 5	From today
Processor	Xtensa LX7, 2 cores, 240 MHz	Cortex-A76, 4 cores, 2.4 GHz
RAM	512 KB, and 8 MB of PSRAM	8 GB
Software	One program in the flash	Linux, Python, many programs
Runtime	TensorFlow Lite Micro	LiteRT, ONNX Runtime, and more
Question	Does the model fit?	How fast does the model run?

- ▶ MobileNetV2 in `int8` did not fit the flash of the XIAO on Day 1. On the Raspberry Pi, its `float32` file of 13.4 MB is a small file.
- ▶ The new budget is the time for one inference.

Key point: On the Raspberry Pi, the model fits. The work of today is to make it fast.

An inference is a long list of multiply-accumulates

```
// One convolution of 3 x 3. Each line "acc += " is one multiply-accumulate (MAC).
for (int oc = 0; oc < C_out; oc++)
  for (int y = 0; y < H; y++)
    for (int x = 0; x < W; x++) {
      float acc = bias[oc];
      for (int ic = 0; ic < C_in; ic++)
        for (int ky = 0; ky < 3; ky++)
          for (int kx = 0; kx < 3; kx++)
            acc += in[ic][y + ky][x + kx] * w[oc][ic][ky][kx];
      out[oc][y][x] = acc;
    }
```

- ▶ MobileNetV2 for an image of 224×224 needs 300.8 million MACs.
- ▶ With one MAC in each clock cycle, the Raspberry Pi 5 needs $300.8 \times 10^6 / (2.4 \times 10^9)$ seconds: 125 ms.
- ▶ A good runtime is much faster than this. You measure the real value in the lab. This part explains where the gain comes from.

Source: A calculation with one MAC in each clock cycle of 2.4 GHz. Number of MACs: the notebook of Day 1.

Three ways to more MACs in one second

SIMD

more values in
one instruction

Threads

more cores work
at the same time

Accelerator

a special unit:
a GPU or an NPU

- ▶ The three ways do the same MACs in less time. They do not change the model.
- ▶ The methods of Days 4 and 6 are the fourth way: fewer MACs, or cheaper MACs.
- ▶ Each way has a limit. The arithmetic units need their data at the right time, and the memory must deliver it.

Key point: Hardware acceleration is parallel arithmetic. Its limit is often the memory, not the arithmetic.

SIMD: one instruction, many values

Scalar: 4 instructions

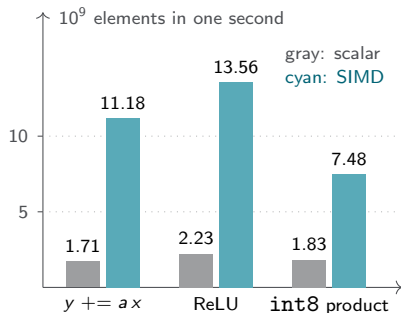
$$\begin{array}{rcl} \boxed{a_0} & + & \boxed{b_0} = \boxed{c_0} \\ \boxed{a_1} & + & \boxed{b_1} = \boxed{c_1} \\ \boxed{a_2} & + & \boxed{b_2} = \boxed{c_2} \\ \boxed{a_3} & + & \boxed{b_3} = \boxed{c_3} \end{array}$$

SIMD: 1 instruction

$$\begin{array}{rcl} \boxed{a_0} & \boxed{a_1} & \boxed{a_2} & \boxed{a_3} & \text{one register} \\ + & \boxed{b_0} & \boxed{b_1} & \boxed{b_2} & \boxed{b_3} \\ \hline = & \boxed{c_0} & \boxed{c_1} & \boxed{c_2} & \boxed{c_3} \end{array}$$

- ▶ SIMD: single instruction, multiple data. A wide register holds some values, and one instruction works on all of them.
- ▶ NEON of the Raspberry Pi: 128 bits. One register holds 4 values of `float32`, or 16 values of `int8`.
- ▶ AVX2 of a laptop processor: 256 bits. 8 values of `float32`, or 32 values of `int8`.
- ▶ The processor reads and decodes one instruction in place of 4. The loop has one quarter of the steps.

Measured: the same loop with and without SIMD



- ▶ The same C code, compiled two times: with and without SIMD instructions.
- ▶ The gain is a factor of 6.5, 6.1, and 4.1.
- ▶ The register holds 8 values of float32. The gain is below 8: the loads and the stores of the data also need time.
- ▶ The arrays have 8 KB. They stay in the first cache.

Key point: You do not write this code. The kernel library of the runtime has a SIMD version of each operator. Part 2 shows the libraries.

Source: An experiment of this course on an x86 processor with AVX2, one thread. The values say nothing about a board.

$$\text{speedup} = \frac{1}{(1 - p) + \frac{p}{G}}$$

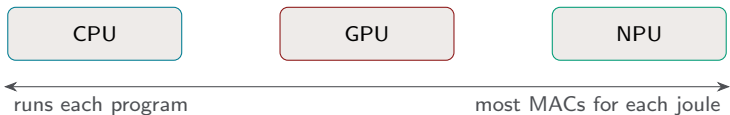
p The part of the time that the faster path does

G The gain on this part: 4 for 4 cores

p	4 cores	No limit on G
0.50	1.6	2
0.80	2.5	5
0.90	3.1	10
0.95	3.5	20

- ▶ A runtime splits the work of a layer between the cores. The Raspberry Pi 5 has 4 cores.
- ▶ Some work stays outside: read the camera, change the size of the image, the Python code, the post-processing.
- ▶ The same law applies to each accelerator. A unit that makes 90 percent of the time 100 times faster gives a speedup of 9.2.

Key point: Measure the complete pipeline, from the camera to the result. The part that you do not accelerate sets the limit.



	CPU	GPU	NPU
Design	A small number of complex cores, with SIMD	Many simple cores that do the same step	A fixed array of MAC units
Runs	Each operator, and all the other code	Programs for large tensors	Only the operators that it supports
Numbers	float32 and int8	Mostly float32 and float16	Mostly int8

- ▶ NPU: neural processing unit. It is a circuit for one kind of work.
- ▶ More special hardware gives more MACs for each joule, and it runs fewer kinds of models.

A CPU core

- ▶ It is made for code with branches.
- ▶ A large part of its area is control logic and cache.
- ▶ A small part does arithmetic.

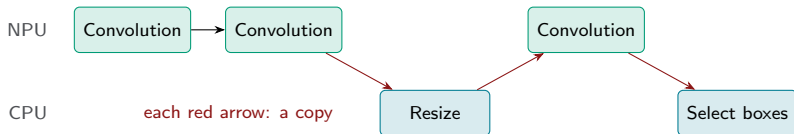
A GPU

- ▶ It is made for one simple step on very many values.
- ▶ Most of its area does arithmetic.
- ▶ One thread is slow. Thousands together are fast.

- ▶ Each call of a GPU has a price: the program sends the data and starts the kernel. For a small model and one image, the CPU can be faster.
- ▶ The Raspberry Pi 5 has a GPU for graphics. The labs run the models on the CPU.

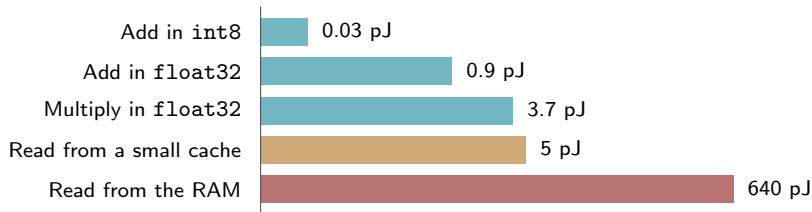
Key point: A GPU pays for large tensors and large batches. An edge device often has one image and a small model.

The NPU: fixed units for one kind of work



- ▶ An NPU has an array of MAC units and a local memory. It does a convolution in `int8` with very little energy.
- ▶ A tool compiles the model for the NPU before the deployment.
- ▶ An operator that the NPU does not support runs on the CPU. This is the **fallback**. The data then moves between the two units, and the gain can go.
- ▶ Examples: the NPU in the chip of a phone, a small NPU next to a microcontroller core, an accelerator board for the Raspberry Pi.

Key point: Before you buy an accelerator, check that it supports each operator of your model.

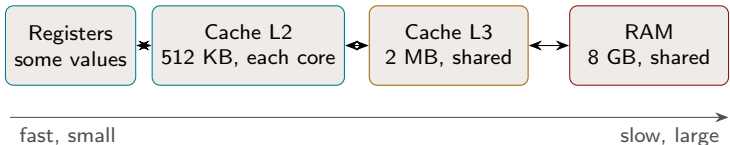


- ▶ The move of a value costs more than the arithmetic. A read from the RAM costs about 170 times the energy of one multiplication. The bars have a logarithmic scale.
- ▶ Time has the same order: the arithmetic units wait for the memory.
- ▶ The gap grows. Each new processor adds more arithmetic than memory bandwidth.

Key point: The question is not only “how many MACs?”. It is also “how many bytes move, and from where?”

Source: *Machine Learning Systems*, Volume I, chapter 11. Estimates of M. Horowitz (2014) for a 45 nm process.

The memory hierarchy

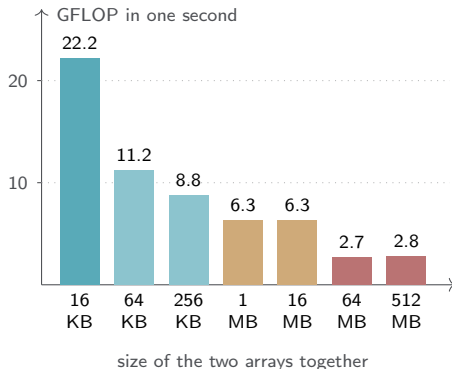


- ▶ The numbers are the sizes of the Raspberry Pi 5. A value in a cache is near the core. A value in the RAM comes over a bus that the 4 cores share.
- ▶ The weights of MobileNetV2: 13.4 MB in `float32`, and 3.3 MB in `int8`. They do not fit in the caches. Each inference reads them from the RAM.
- ▶ Its first feature map, $112 \times 112 \times 32$: 1.5 MB in `float32`, and 0.4 MB in `int8`.

Key point: A smaller tensor is faster two times: fewer bytes move, and they stay in a nearer memory.

Source: Cache sizes: product brief of the Raspberry Pi 5. Tensor sizes: a calculation with 3 504 872 parameters.

Measured: one loop, and data of different sizes



- ▶ The loop is $y += ax$ with SIMD: one MAC for each element.
- ▶ Caches of this processor: 32 KB, 256 KB, and 30 MB.
- ▶ Each step down in the plot is one cache that is too small.
- ▶ From the first cache to the RAM: a factor of 8.

Key point: The same code and the same arithmetic. Only the place of the data changes.

Source: An experiment of this course on an x86 processor with AVX2, one thread. The values say nothing about a board.

Compute-bound

- ▶ The arithmetic units are busy all the time.
- ▶ The data arrives fast enough.
- ▶ A faster core, wider SIMD, or more cores help.

Memory-bound

- ▶ The arithmetic units wait for data.
 - ▶ The memory bus is busy all the time.
 - ▶ Only fewer bytes, or a faster memory, help.
-
- ▶ One model has layers of the two types. A large convolution does many MACs with each byte. A ReLU does one comparison and moves 8 bytes.
 - ▶ The wrong optimization gives no gain. More cores do not help a workload that waits for the memory.

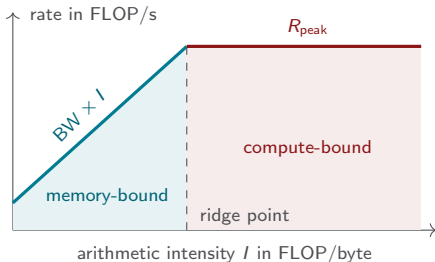
Key point: First find the limit of the workload. Then select the optimization.

$$I = \frac{\text{operations}}{\text{bytes that move}} \quad \text{unit: FLOP for each byte}$$

Operator in float32	I
Add two feature maps	0.083
ReLU	0.125
Fully connected layer, one input vector	0.5
Depthwise convolution 3×3	2.25
Convolution 3×3 , 64 to 64 channels, 56×56	132

- ▶ One MAC is 2 operations. The bytes are the input, the weights, and the output: each tensor one time.
- ▶ A fully connected layer uses each weight one time. A convolution uses each weight at each position of the image.
- ▶ In `int8`, the same operator moves one quarter of the bytes. Its intensity is 4 times as high.

Source: A calculation. Method: *Machine Learning Systems*, Volume I, chapter 11.



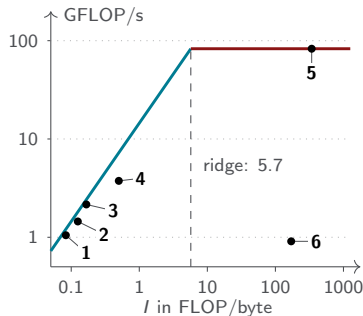
$$R = \min(R_{peak}, BW \times I)$$

- R The highest rate that the workload can reach
- R_{peak} The peak rate of the arithmetic units
- BW The memory bandwidth in bytes for each second

- ▶ The two axes have a logarithmic scale. The roof has two parts: a slope and a flat part.
- ▶ The **ridge point** is R_{peak}/BW . A workload with a lower intensity is memory-bound.
- ▶ The hardware gives the roof. The workload gives the position below the roof.

Source: Williams, Waterman, and Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures", 2009.

Measured: the roofline of one core



	Workload	I	Rate	Roof
1	Add two arrays	0.083	1.05	1.20
2	ReLU	0.125	1.45	1.81
3	$c = a + 3b$	0.167	2.16	2.41
4	Matrix $\times$ vector	0.5	3.75	7.23
5	Product, library	341	82.8	82.8
6	Product, three loops	171	0.91	82.8

- ▶ Peak: 82.8 GFLOP/s (workload 5). Bandwidth: 14.5 GB/s (a read loop).
- ▶ Workloads 1 to 4 use arrays that are much larger than the caches. They reach 52 to 90 percent of the roof. Rates in GFLOP/s.
- ▶ Workload 4 is a fully connected layer with one input vector. Its rate is 22 times lower than the rate of workload 5.

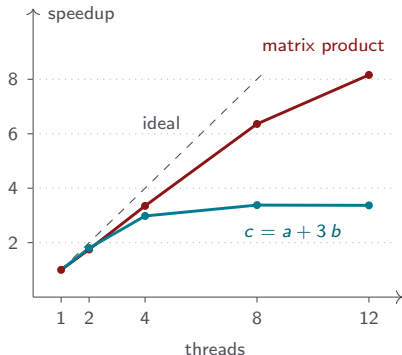
Source: An experiment of this course on an x86 processor with AVX2, one thread.

Fact	Example of the measurement
The roof is a limit. It is not a prediction.	The matrix product with three simple loops reaches 0.91 GFLOP/s. The library reaches 82.8 with the same arithmetic: a factor of about 90.
The intensity depends on the code.	The library cuts the matrices into blocks that fit in a cache. It uses each byte many times before the byte leaves the cache.
The bandwidth depends on the place of the data.	The loop $y += ax$ has an intensity of 0.167. With arrays in the first cache it reaches 22.2 GFLOP/s. The roof for data in the RAM is 2.4.

- ▶ A very small model has one more limit: the fixed cost of each operator in the runtime.

Key point: The roof tells you which resource limits a good kernel. The quality of the kernel decides how near to the roof you get.

Source: The experiment of this course: an x86 processor with AVX2, one thread.



- ▶ The matrix product is compute-bound. With 12 threads it is 8.2 times as fast.
- ▶ The loop $c = a + 3b$ is memory-bound. Its gain stops at 3.4.
- ▶ All cores share the memory. The bandwidth grows from 14.5 to 59.6 GB/s with 8 threads, and then it stops.

Key point: A test for your model: change the number of threads. A small gain is a sign of a memory-bound workload, or of a part that is not parallel.

Source: The experiment of this course: an x86 processor with 12 cores and AVX2. The values say nothing about a board.

	Memory-bound	Compute-bound
Typical operators	ReLU, add, a fully connected layer, a depthwise convolution	A large convolution, a matrix product with a batch
What helps	Fewer bytes: <code>int8</code> , smaller feature maps. Fused operators (Part 3).	SIMD kernels, threads, an accelerator. Fewer MACs (Day 6).
No help	More cores, a higher peak rate	A faster memory

- ▶ `int8` helps in the two regimes: one quarter of the bytes, and 4 times as many values in one register.
- ▶ This is the limit of the hardware. The kernels decide what you get: on the work computer of Day 6, the `int8` models were 1.6 times slower.
- ▶ A language model that makes one token reads all its weights for this token. It is memory-bound (Day 10).

Processor	4 cores of the type Cortex-A76, 2.4 GHz
SIMD	NEON: 128 bits, 2 SIMD units in each core
Peak, float32	$2 \text{ units} \times 4 \text{ values} \times 2 \text{ operations} \times 2.4 \text{ GHz}$: 38.4 GFLOP/s for one core, 153.6 GFLOP/s for 4 cores
Caches	512 KB for each core (L2), 2 MB for all cores (L3)
RAM	8 GB. The 4 cores share its bandwidth.
GPU, NPU	A GPU for graphics. No NPU: an accelerator board can use the PCIe port.

- ▶ The same calculation for the work computer gives 80 GFLOP/s (2 units, 8 values, 2 operations, 2.5 GHz). The measurement: 82.8.

Key point: On this board, acceleration is: NEON kernels, 4 threads, and fewer bytes. The runtime gives the first two. Parts 2 and 3 show how.

Source: Product brief of the Raspberry Pi 5. SIMD units: documentation of Arm for the Cortex-A76. The peak is a calculation.

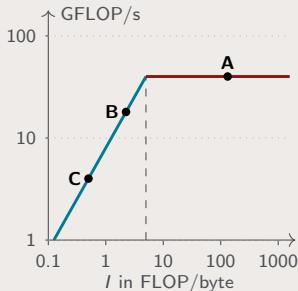
Exercise: place three workloads on a Roofline plot

One core of an example device has a peak rate of 40 GFLOP/s and a memory bandwidth of 8 GB/s. Three layers in float32 run on it. The table gives the operations and the bytes in millions.

Layer	Operations	Bytes
A Convolution 3×3 , 64 to 64 channels, 56×56	231.2	1.75
B Depthwise convolution 3×3 , 32 channels, 112×112	7.23	3.21
C Fully connected, 1280 to 1000	2.56	5.13

1. Calculate the ridge point of the device. Draw the roof: the axes are the intensity from 0.1 to 1000, and the rate from 1 to 100 GFLOP/s.
2. Calculate the arithmetic intensity of each layer. Place the three layers on the plot, at the roof.
3. Which layers are memory-bound? Calculate the shortest time of each layer.
4. The model changes to int8: each layer moves one quarter of the bytes. Which layer changes its regime?

Answer: place three workloads on a Roofline plot



1. Ridge point: $40/8 = 5$ FLOP for each byte.
2. A: $231.2/1.75 = 132$. B: $7.23/3.21 = 2.25$. C: $2.56/5.13 = 0.5$.
3. B and C are below 5: memory-bound. Rates: A 40, B $8 \times 2.25 = 18$, C $8 \times 0.5 = 4$ GFLOP/s. Times: A 5.8 ms, B 0.40 ms, C 0.64 ms.
4. The intensities become 528, 9, and 2. B goes over the ridge point: it is now compute-bound. C stays memory-bound. Its limit grows to 16 GFLOP/s.

Layer C has one third of the operations of layer B, and it needs more time. A count of operations alone does not give the time. For question 4, the peak rate of the device stays at 40.

Part 1 Acceleration fundamentals

Exercise: place three workloads on a Roofline plot

Part 2 The runtime stack

Exercise: select a runtime for three deployment cases

Part 3 Graph optimization

Exercise: fuse a convolution, a batch normalization, and a ReLU on paper

Why not PyTorch on the device?

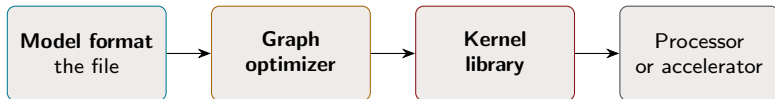
	Training framework	Inference runtime
Work	Forward pass, gradients, updates	The forward pass only
Graph	Python code makes it at each call	Fixed. It comes from a file.
Installed size	733 MB (PyTorch)	21 to 57 MB
Load the model	2.8 s	0.05 to 0.11 s
One inference	31.9 ms	11.7 to 16.6 ms

- ▶ A training framework can do each operation on each tensor. This freedom costs time and memory.
- ▶ A runtime knows the complete graph before the first inference. It can prepare the work.

Key point: Train in the framework. Export the model one time. Run the exported file in a runtime.

Source: MobileNetV2 with one thread in ONNX Runtime, LiteRT, and NCNN: an experiment of this course on an x86 processor with AVX2.

The three layers of a runtime



What is the model?

What work is left?

How is it done?

- ▶ **Model format:** the graph, the operators, and the weights in one file.
- ▶ **Graph optimizer:** it changes the graph before the first inference. The result of the model stays the same. Part 3 gives the details.
- ▶ **Kernel library:** the code of each operator for one type of processor. A kernel uses the SIMD instructions and the threads of Part 1.

Key point: Compare two runtimes layer by layer: which file, which optimizations, which kernels.

Layer 1: the model format

Runtime	File	Made by	MobileNetV2
LiteRT	<code>.tflite</code>	The LiteRT converter	14 078 620
ONNX Runtime	<code>.onnx</code>	<code>torch.onnx.export</code>	14 216 752
NCNN	<code>.param</code> and <code>.bin</code>	The tool <code>pnnx</code>	7 020 315
ExecuTorch	<code>.pte</code>	The ExecuTorch export	13.58 MB

- ▶ The sizes are in bytes. The model has 3 504 872 parameters: 14 019 488 bytes in `float32`. The graph adds a small part.
- ▶ The NCNN file is half as large: the tool stores the weights in `float16`.
- ▶ A `.tflite` file is the FlatBuffer of Day 3. An `.onnx` file has a different container, and it holds the same three things.

Source: File sizes: an export of this course from one PyTorch model. ExecuTorch file: the guide “Image Classification with EXECUTORCH” of *EdgeML with Raspberry Pi*.

```
import torch, torchvision

model = torchvision.models.mobilenet_v2(weights="IMAGENET1K_V1").eval()
example = torch.randn(1, 3, 224, 224)

torch.onnx.export(model, (example,), "mnv2_static.onnx",
                  input_names=["image"], output_names=["logits"],
                  dynamo=True, external_data=False)
```

- ▶ ONNX is an open standard for model files. It defines a list of operators. The list has a version number: the **opset**.
- ▶ The export follows the example input through the model and writes the graph. The model must be in the mode `eval()`.
- ▶ This file has 100 nodes: 52 Conv, 35 Clip, 10 Add, and 3 more. The opset is 20.
- ▶ Many tools read ONNX. So ONNX is also a step on the way to a different format.

Key point: Open each exported file in Netron. Check the input shape, the operators, and the output.

When	Who	Example
At the export	The converter on the laptop	The LiteRT converter, <code>pnnx</code>
When the program loads the model	The runtime on the device	ONNX Runtime

- ▶ An optimization at the export costs no time on the device. The file is then made for one runtime.
- ▶ An optimization at the load can use facts about the device. It makes the start of the program slower.
- ▶ ONNX Runtime with MobileNetV2 and one thread: 26.4 ms with no graph optimization, and 11.7 ms with all optimizations. A part of this gain exists only on an x86 processor.

Key point: One file gives a different latency at each optimization level.

Source: An experiment of this course on an x86 processor with AVX2.

A reference kernel

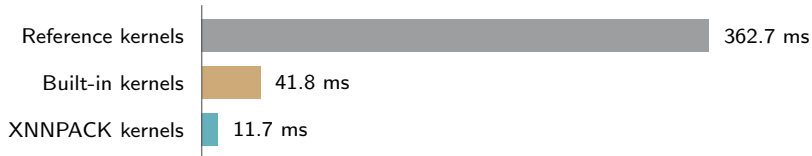
- ▶ Simple loops, as the listing of Part 1.
- ▶ It is easy to read and to test.
- ▶ It is the same code for each processor.

An optimized kernel

- ▶ SIMD instructions of one processor type.
- ▶ Blocks of data that fit in a cache.
- ▶ Work for more than one thread.

Library	Used by
XNNPACK	LiteRT (the default on a CPU), ExecuTorch, and ONNX Runtime as an option
MLAS	ONNX Runtime (the default on a CPU)
NCNN	Its own kernels, written first for the Arm processors of phones
CMSIS-NN	TensorFlow Lite Micro on Arm Cortex-M (Day 3)

Measured: one file, three sets of kernels



- ▶ One `.tflite` file of MobileNetV2 in LiteRT, with one thread. Only the kernels change.
- ▶ From the reference kernels to XNNPACK: a factor of 31.
- ▶ The three runs give the same class.
- ▶ LiteRT selects XNNPACK with no setting. When it starts, it prints Created TensorFlow Lite XNNPACK delegate for CPU.

Key point: The kernel library alone can change the latency by a factor of 30.

Source: An experiment of this course on an x86 processor with AVX2. The values say nothing about a board.

MobileNetV2 in ExecuTorch	File	Latency
float32, kernels with no optimization	13.58 MB	2440 ms
float32, XNNPACK	13.35 MB	10.84 ms
int8, XNNPACK	3.59 MB	3.69 ms

- ▶ The latency is the median of 50 runs on a Raspberry Pi 5. With XNNPACK, the same model is 225 times as fast.
- ▶ The calculation of Part 1 with one MAC in each clock cycle gave 125 ms. XNNPACK is more than 11 times faster than this.
- ▶ `int8` is 2.9 times as fast as `float32`. On the work computer of Day 6, `int8` was slower. The processor and its kernels make the difference.

Key point: An export with no optimized backend runs. You cannot use its latency.

Source: The guide “Image Classification with EXECUTORCH” of *EdgeML with Raspberry Pi*, by M. Rovai.

```
from ai_edge_litert.interpreter import Interpreter

interpreter = Interpreter(model_path="mnv2.tflite", num_threads=4)
interpreter.allocate_tensors()
inp = interpreter.get_input_details()[0]
out = interpreter.get_output_details()[0]

interpreter.set_tensor(inp["index"], image)
interpreter.invoke()
scores = interpreter.get_tensor(out["index"])
```

- ▶ LiteRT is the new name of TensorFlow Lite. TensorFlow Lite Micro of Day 3 is its version for microcontrollers.
- ▶ It is an interpreter: it reads the .tflite file and calls one kernel for each operator.
- ▶ `allocate_tensors()` plans the memory of all tensors one time, as the arena of Day 3.
- ▶ A **delegate** takes a part of the graph. XNNPACK is the default delegate for the CPU.

```
import onnxruntime as ort

options = ort.SessionOptions()
options.intra_op_num_threads = 4
options.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL

session = ort.InferenceSession("mnv2_static.onnx", options,
                               providers=["CPUExecutionProvider"])
scores = session.run(None, {"image": image})[0]
```

- ▶ ONNX Runtime runs .onnx files from each framework that can export ONNX.
- ▶ It optimizes the graph when it makes the session. The level is a setting.
- ▶ An **execution provider** is its name for a backend. The list providers gives the order. The CPU provider can run each operator.
- ▶ One file runs on a laptop, on a server, and on the Raspberry Pi.

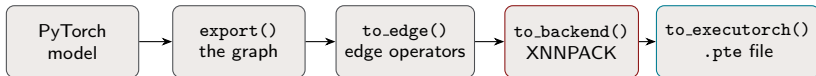
What it is

- ▶ A runtime that was made for phones.
- ▶ It has its own kernels for Arm processors with NEON.
- ▶ It needs no other library.
- ▶ Two files: `.param` is the graph as text, and `.bin` holds the weights.
- ▶ The export fuses each convolution with its activation. The graph of MobileNetV2 has 77 layers.
- ▶ The kit lab for object detection reports the best speed on the Raspberry Pi for the NCNN format. You measure this on Day 8.

How you get a model

- ▶ From PyTorch with the tool `pnnx`.
- ▶ From a training tool that exports NCNN. The YOLO tool of Day 8 does this with one command.

Source: *Machine Learning Systems*, hardware kits, Raspberry Pi, “Object Detection”. Number of layers: an export of this course.



- ▶ ExecuTorch is the runtime of PyTorch for edge devices. The model stays in the PyTorch tools from the training to the device.
- ▶ All steps run on the laptop, before the deployment. The runtime on the device is small: it reads the .pte file and runs the plan.
- ▶ The step `to_backend()` gives the parts of the graph that XNNPACK supports to XNNPACK. Without this step, the file uses kernels with no optimization.
- ▶ It is newer than LiteRT (2023 against 2017). LiteRT has more documents and more ready models.

Source: The guide “Image Classification with EXECUTORCH” of *EdgeML with Raspberry Pi*, by M. Rovai.

Measured: one model in four programs

Median latency in ms	1 thread	2 threads	4 threads	Gain	Load
PyTorch, no export	31.9	26.7	19.8	1.6	2.80 s
ONNX Runtime	11.7	6.3	3.6	3.3	0.11 s
LiteRT	11.7	6.4	3.5	3.4	0.05 s
NCNN	16.6	9.8	5.8	2.9	0.05 s

- ▶ ONNX Runtime and LiteRT are equal on this processor. They are 2.7 times as fast as PyTorch.
- ▶ The kernels of NCNN are made for Arm, not for this x86 processor. On the Raspberry Pi, the order can change.
- ▶ The gain from 1 thread to 4 threads is near 3, not 4. Possible reasons: work that is not parallel, and memory-bound layers.

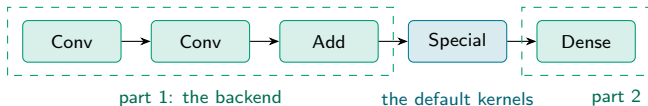
Key point: This table is valid for one model on one processor. In the lab you make the same table on the Raspberry Pi.

Source: MobileNetV2 on an x86 processor: an experiment of this course.

Runtime	Setting	With no setting
LiteRT	<code>Interpreter(num_threads=4)</code>	1 thread
ONNX Runtime	<code>options.intra_op_num_threads = 4</code>	All cores
NCNN	<code>net.opt.num_threads = 4</code>	All cores

- ▶ The threads of this table split the work of one operator.
- ▶ Do not use more threads than free cores. In the experiment, NCNN got 4 cores of a computer with 24 cores and no setting. It needed 20.1 ms. With one thread it needed 16.6 ms.
- ▶ LiteRT with no setting needed 11.6 ms: the time of one thread.
- ▶ Your program needs a core for the camera and for the pre-processing. Four threads for the model can make the complete pipeline slower.
- ▶ Four busy cores make more heat on the Raspberry Pi (Day 9).

Source: An experiment of this course on an x86 processor with AVX2.



- ▶ A backend is a kernel library or an accelerator. It supports a list of operators.
- ▶ The runtime cuts the graph into parts. A part with supported operators goes to the backend. The other operators run with the default kernels.
- ▶ The names: a **delegate** in LiteRT, an **execution provider** in ONNX Runtime, a **backend** in ExecuTorch.
- ▶ Each border between two parts costs time. One operator with no support in the middle of the graph makes two borders.
- ▶ On the Raspberry Pi 5, the backend is a CPU library. With an NPU, the same method sends the parts to the NPU.

The same model, the same result?

Runtime	Weights	Largest difference
ONNX Runtime	float32	0.00007 percent
LiteRT	float32	0.00011 percent
NCNN	float16	1.0 percent

- ▶ For one input, the table compares the 1000 outputs with the outputs of PyTorch. The unit is the largest output.
- ▶ Two float32 runtimes do the additions in a different order. The rounding gives a small difference.
- ▶ A lower precision gives a larger difference. Here, all runtimes give the same class.
- ▶ After each conversion, measure the accuracy on the test set, as on Day 4. One input is not a test.

Key point: A conversion can change the result. A model that loads with no error is not a model that is correct.

Source: An experiment of this course with MobileNetV2.

Runtime	A good first choice when
LiteRT	The model comes from Keras or TensorFlow. The same .tflite family also runs on a microcontroller.
ONNX Runtime	The model comes from PyTorch, and one file must run on different computers.
NCNN	The target is an Arm processor, and the training tool exports NCNN.
ExecuTorch	The team works with PyTorch and wants one tool from the training to the device.

The table gives candidates. Answer these questions, and then measure on the device:

1. Does the runtime have an optimized kernel for each operator of the model, on this processor?
2. Does it support the precision that you need: int8, float16?
3. Does it support the accelerator of the product?
4. Can your program use it: Python, C++, the size of the install?

Exercise: select a runtime for three cases

- A A team trains an image classifier in PyTorch. The first target is a Raspberry Pi 5 with a Python program. Later, the same model file must run on a laptop with an x86 processor.
- B A keyword model comes from Keras, in `int8`. It runs on a microcontroller with TensorFlow Lite Micro. The team wants the same model on a Raspberry Pi that collects the results.
- C An object detector must run on a Raspberry Pi 5 with a camera. The frame rate is the main goal. The training tool exports ONNX, LiteRT, and NCNN.

For each case:

1. Select one runtime. Give the reason with the three layers: format, graph optimizer, kernel library.
2. Name one fact that you check before the decision is final.
3. Give the number of threads for the model, and the reason.

Answer: select a runtime for three cases

	Runtime	Reason	Check
A	ONNX Runtime	PyTorch exports ONNX. One file runs on Arm and on x86, with optimized kernels.	The accuracy of the exported file
B	LiteRT	The .tflite file of the microcontroller runs with no new conversion.	The same class on the two devices for test inputs
C	NCNN first	Its kernels are made for Arm. The tool exports it with one command.	The frame rate and the accuracy of the three formats on the board

- ▶ ExecuTorch with XNNPACK is a second good answer for case A, if the laptop is not a hard requirement.
- ▶ Threads: 4 for a model that runs alone (A). For case C, compare 3 and 4 threads on the complete pipeline, because the camera needs a core. For case B, the model is small: start with 1.

Part 1 Acceleration fundamentals

Exercise: place three workloads on a Roofline plot

Part 2 The runtime stack

Exercise: select a runtime for three deployment cases

Part 3 Graph optimization

Exercise: fuse a convolution, a batch normalization, and a ReLU on paper

You do not run the graph that you trained

MobileNetV2	Size of the graph
The PyTorch model	141 layers
ONNX file, export with no optimization	560 nodes
ONNX file, normal export	100 nodes
The same file in ONNX Runtime, after all optimizations	56 nodes
LiteRT file	70 operators
NCNN file	77 layers

- ▶ The PyTorch model has 52 convolutions, 52 batch normalizations, 35 ReLU6 activations, 1 dropout, and 1 fully connected layer.
- ▶ All these graphs give the same class for an image. Their outputs differ only by the rounding.
- ▶ The optimizer removes work. It does not remove information.

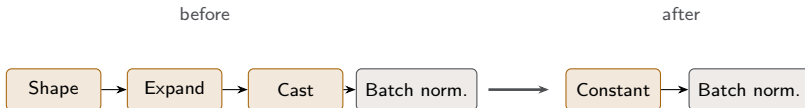
Key point: An optimizer changes the graph before the first inference. After that, each inference does less work.

Source: An export of this course from one PyTorch model.

Six jobs of a graph optimizer

Job	What it does	The result
Constant folding	It calculates each value that never changes, one time	The same
Removal of unused nodes	It removes each node that no output needs	The same
Operator fusion	It makes one operator from operators in a row	The same
Memory layout	It puts the values of a tensor in the order that the kernels want	The same
Shape planning	It calculates the size of each tensor and plans the memory	The same
Precision selection	It changes float32 to float16 or to int8	It changes

- ▶ “The same” means: the same to about 6 digits. The order of the additions changes, so the rounding changes.
- ▶ Only the last job needs data: a calibration set, and a test of the accuracy.



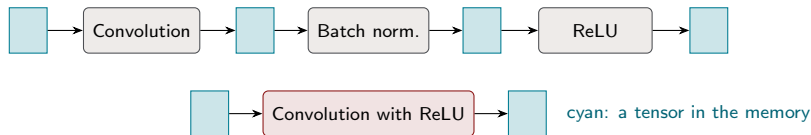
- ▶ A node with only constant inputs gives the same output at each inference. The optimizer runs the node one time and stores the output.
- ▶ The export of MobileNetV2 with no optimization has 560 nodes. Most of them prepare the parameters of the batch normalizations: 127 Constant, 122 CastLike, 104 Expand, and 52 Shape.
- ▶ The optimizer of the exporter folds these nodes. With the fusion of the next frames, 100 nodes stay.
- ▶ With a static input shape, each calculation of a shape is also a constant.

Source: An export of this course with `torch.onnx.export`, with and without the option `optimize`.

Node	Why it can go
Dropout	In an inference, it gives its input as its output.
Identity	It only copies a tensor.
A node for the training	The loss, or a second output that only the training reads. No output of the deployed model needs it.
Two equal subgraphs	They have the same inputs and the same operators. One of them is enough.

- ▶ The optimizer starts at the outputs and goes back through the graph. A node that it does not reach has no reader.
- ▶ Put the model in the mode `eval()` before the export. A model in the training mode keeps its dropout, and its batch normalizations use the statistics of the batch.

Key point: A node that no output needs costs time at each inference. After the export, check that the file has only the nodes of the inference.



- ▶ Each operator reads its input from the memory and writes its output to the memory. Three operators make three passes.
- ▶ The batch normalization and the ReLU do very little arithmetic for each byte. They are memory-bound (Part 1).
- ▶ The fused operator makes one pass. The values between the steps stay in a register.
- ▶ The first layer of MobileNetV2 in `float32`: the three operators move 8.2 MB. The fused operator moves 2.1 MB.

Key point: Fusion removes complete passes over the memory. It is the most important optimization for memory-bound operators.

Source: A calculation: an input of $224 \times 224 \times 3$ and a feature map of $112 \times 112 \times 32$, each tensor one time.

Fold the batch normalization into the convolution

In an inference, the two operators are:

$$z = W * x + b$$
$$y = \gamma \frac{z - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$$

One convolution gives the same y :

$$s = \frac{\gamma}{\sqrt{\sigma^2 + \varepsilon}}$$
$$W' = s W \quad b' = s(b - \mu) + \beta$$

- ▶ γ and β are trained. μ and σ^2 are the mean and the variance that the training collected. After the training, all four are constants.
- ▶ Each output channel has its own s .
- ▶ The check on the first layer of MobileNetV2: the largest difference between the two outputs is 9.5×10^{-7} . The largest output is 3.1.
- ▶ The model loses 68 224 values of its batch normalizations and gets 17 056 bias values.

Key point: The folding is constant folding with algebra. After it, the batch normalization costs nothing.

Source: Formulas: *Machine Learning Systems*, Volume I, chapter 13. Check and counts: an experiment of this course.

How

- ▶ ReLU is $\max(0, y)$. ReLU6 limits y to the range from 0 to 6.
- ▶ The kernel of the convolution applies the limit before it writes each output value.
- ▶ In `int8`, the limit is the limit of the integer range. It costs nothing.

How you see it

- ▶ LiteRT: the operator `CONV_2D` has the property “fused activation”.
- ▶ ONNX Runtime: the node `FusedConv`.
- ▶ NCNN: the convolution layer has the activation as a parameter.

- ▶ The LiteRT file of MobileNetV2 from the export of this course has no activation operator. All 35 are in the convolutions.
- ▶ A fusion is possible only when one operator reads the middle tensor. A tensor with two readers must exist in the memory.
- ▶ Some runtimes also fuse the addition of a residual connection into the convolution before it.

Measured: optimization levels of ONNX Runtime

Level	Nodes	What changes	Latency	Load
Disabled	152	Nothing	27.7 ms	0.067 s
Basic	100	The 52 batch normalizations go into the convolutions	26.1 ms	0.078 s
Extended	65	35 activations go into the convolutions	25.2 ms	0.086 s
All	56	A memory layout for the SIMD kernels. The 10 additions go into the convolutions.	11.5 ms	0.097 s

- ▶ The file is an export of MobileNetV2 that keeps its batch normalizations.
- ▶ The two fusion steps give 9 percent. The memory layout gives a factor of 2.2 on this processor.
- ▶ The output of each level differs from the first output by 0.0001 percent of the largest value. The load needs 0.03 s more.

Source: An experiment of this course on an x86 processor, one thread.



- ▶ The figure shows an image of 2×2 pixels with 3 channels in the memory. The same 12 values, in two orders.
- ▶ PyTorch and ONNX use NCHW. LiteRT and XNNPACK use NHWC. A SIMD kernel is fast when the values that it needs lie side by side.
- ▶ ONNX Runtime on x86 uses a third layout: blocks of 8 channels, which fill one AVX2 register. This is the last level of the table before.
- ▶ Where two layouts meet, the converter adds a Transpose operator. The LiteRT file from PyTorch has one at its input.

Key point: Check the input shape of each file in Netron: $1 \times 3 \times 224 \times 224$ or $1 \times 224 \times 224 \times 3$. Your pre-processing must give this order.

With all shapes known

- ▶ The size of each tensor is a constant.
- ▶ The runtime plans the memory before the first inference: one arena, as on Day 3.
- ▶ Two tensors that never live at the same time share the same bytes.
- ▶ In LiteRT, the call `allocate_tensors()` does this plan. For MobileNetV2 it needs 10 ms, one time.
- ▶ Some backends accept only static shapes. An NPU is the usual example.

What you gain

- ▶ No memory allocation during an inference.
- ▶ A peak memory that you know before the deployment.
- ▶ A latency that changes little from run to run.

Source: Time of the plan: an experiment of this course on an x86 processor.

```
batch = torch.export.Dim("batch", min=1, max=64)

torch.onnx.export(model, (example,), "mnv2_dynamic.onnx",
                  input_names=["image"], output_names=["logits"],
                  dynamic_shapes={"x": {0: batch}},
                  dynamo=True, external_data=False)
```

- ▶ A dimension of the input gets a name in place of a number. Netron shows the input shape $\text{batch} \times 3 \times 224 \times 224$.
- ▶ The key "x" is the name of the input in the function forward of the model.
- ▶ The static file accepts only its shape. With 2 images, ONNX Runtime stops with Got: 2 Expected: 1.
- ▶ The usual dynamic dimensions: the batch size, the size of the image, the length of a text.

Key point: A dynamic dimension is a decision at the export. The runtime cannot add it later.

ONNX Runtime	ms
Static file, 1 image	11.7
Dynamic file, 1 image	11.3
Dynamic file, 4 images, first run	57.8
Dynamic file, 4 images, next runs	46.1

Dynamic image size	ms
128×128	4.3
160×160	6.2
192×192	8.7
224×224	11.8

- ▶ For one image, the dynamic file costs nothing here.
- ▶ The first run with a new shape is 26 percent slower: the runtime plans the memory again. LiteRT needs 15 ms for a new plan.
- ▶ A batch of 4 needs 11.5 ms for each image: no gain on one core. A smaller image is faster: the time follows the number of pixels.

Key point: On an edge device, start with a static shape and a batch of 1. Use a dynamic dimension only when the product needs it.

Source: An experiment of this course on an x86 processor.

LiteRT file of MobileNetV2	Bytes	Accuracy	Latency
float32	14 078 620	77.4	11.7 ms
Weights in int8, the rest in float32	3 918 032	77.6	11.9 ms
Dynamic range	3 887 696	77.6	28.5 ms
Full int8, with calibration	4 107 968	75.2	16.0 ms

- ▶ Accuracy in percent on 500 images of 10 ImageNet classes.
- ▶ Each int8 file has 28 to 29 percent of the size of the float32 file.
- ▶ Full int8 loses 2.2 points here. Is this acceptable? The requirement of the product decides.
- ▶ The latency is from an x86 processor: int8 gives no gain there. On the Raspberry Pi 5, the published gain is a factor of 2.9 (Part 2).

Key point: The precision is a decision with three numbers: size, accuracy, and latency. Measure the latency on the target.

Source: An experiment of this course with the validation images of Imagenette, on an x86 processor with AVX2.

```
from onnxruntime.quantization import (CalibrationDataReader, QuantFormat,
                                     QuantType, quantize_static)

class Reader(CalibrationDataReader):
    def __init__(self, images):
        self.items = iter([{"image": x[None]} for x in images])
    def get_next(self):
        return next(self.items, None)

quantize_static("mnv2_prepared.onnx", "mnv2_int8.onnx", Reader(images),
               quant_format=QuantFormat.QDQ, per_channel=True,
               activation_type=QuantType.QUInt8, weight_type=QuantType.QInt8)
```

- ▶ The tool is a part of the Python package of ONNX Runtime. Its function `quant_pre_process` makes the prepared file.
- ▶ The reader gives the calibration images one by one, with the pre-processing of the model. The tool runs the model and records the range of each activation (Day 4).
- ▶ The result is an ONNX file with `QuantizeLinear` and `DequantizeLinear` nodes. The runtime fuses them into `int8` operators.

Calibration set	ONNX Runtime	LiteRT
No quantization: float32	77.4	77.4
100 training images, range from the minimum and the maximum	71.0	75.2
100 training images, range from a percentile	74.4	not measured
100 images of random noise	14.4	12.8

- ▶ Accuracy in percent of the int8 file, on 500 validation images. The calibration images come from the training set.
- ▶ With noise, the tool records ranges that no real image gives. The model is then almost random.
- ▶ The method for the range also counts: 3.4 points between two settings of one tool.

Key point: The same rule as on Day 4: calibrate with data that looks like the data of the product. Then test the accuracy.

1. **Look.** Open the file before and after in Netron. Find the operators that are gone and the operators that are new.
2. **Count.** Write the number of operators of each type.
3. **Compare.** Give the two files the same input. The outputs must agree to about 6 digits for `float32`.
4. **Measure.** The latency, the load time, and for a new precision the accuracy.

Runtime	Where you see the optimized graph
LiteRT, NCNN	In the exported file: the converter did the work.
ONNX Runtime	Set <code>options.optimized_model_filepath</code> . The runtime then writes the graph that it runs.

Key point: Do not trust a setting. Look at the graph that runs.

	Part	Time	You do
A	Setup	35 min	Start the Raspberry Pi, connect with SSH, and test the camera.
B	First inference	35 min	Run MobileNetV2 with LiteRT on test images and on the camera.
C	Export and inspect	45 min	Export one PyTorch model to ONNX with a static shape and with a dynamic shape, and to LiteRT. Find the fused operators in Netron.
D	Measure	35 min	Measure the latency for each runtime, thread count, and precision.

- ▶ You hand in the latency table and one Decision Log entry.
- ▶ The check: the table has two runtimes or more, two thread counts or more, and two precisions or more.
- ▶ Before Part D, write your prediction for each row. The numbers of the lecture are from an x86 processor. Your table gives the numbers of the Raspberry Pi.

Exercise: fuse three operators on paper

A convolution has one filter with three weights and a bias. Then come a batch normalization and a ReLU. Ignore ε .

Convolution	$W = (2, -1, 0.5)$ and $b = 1$
Batch normalization	$\gamma = 3, \beta = -1, \mu = 3, \sigma = 2$

1. Calculate s , the weights W' , and the bias b' of the fused convolution.
2. The input is $x = (2, 1, 4)$. Calculate the output with the three operators, and then with the fused operator.
3. Do the same for $x = (0, 2, 2)$. Where is the ReLU in the fused operator?
4. The input tensor has 0.57 MB and each later tensor has 1.53 MB. Each operator reads its input and writes its output. How many MB move with three operators, and with the fused operator?

1. $s = \gamma/\sigma = 1.5$. So $W' = (3, -1.5, 0.75)$ and $b' = 1.5 \times (1 - 3) - 1 = -4$.
2. Three operators: $z = 4 - 1 + 2 + 1 = 6$. Then $y = 3 \times (6 - 3)/2 - 1 = 3.5$. The ReLU keeps 3.5. Fused: $6 - 1.5 + 3 - 4 = 3.5$.
3. Three operators: $z = 0 - 2 + 1 + 1 = 0$. Then $y = 3 \times (0 - 3)/2 - 1 = -5.5$. The ReLU gives 0. Fused: $0 - 3 + 1.5 - 4 = -5.5$, and the limit at the end of the kernel gives 0. The ReLU is one comparison before the kernel writes the value.
4. Three operators: $(0.57 + 1.53) + (1.53 + 1.53) + (1.53 + 1.53) = 8.22$ MB. Fused: $0.57 + 1.53 = 2.10$ MB. The fusion removes 74 percent of the bytes that move.

The fused operator has 4 values in place of 8. The result is the same for each input.

- ▶ A CPU, a GPU, and an NPU get their speed from parallel arithmetic. The memory must deliver the data at the same rate.
- ▶ The Roofline model gives the limit of a workload: the peak compute rate, or the memory bandwidth times the arithmetic intensity.
- ▶ A runtime has three layers: the model format, the graph optimizer, and the kernel library. Two runtimes can give different latencies for the same model.
- ▶ A graph optimizer folds constants, removes unused nodes, and fuses operators. With a static input shape, the runtime plans the memory one time.

Key point: The latency is a property of the model, the runtime, and the device together. Measure each combination on the Raspberry Pi before you select one.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I: chapter 11 “Hardware Acceleration” and chapter 13 “Model Serving”.
- ▶ *Machine Learning Systems*, Raspberry Pi labs: “Setup” and “Image Classification”.

Papers

- ▶ “Roofline: An Insightful Visual Performance Model for Multicore Architectures”, Williams, Waterman, and Patterson, 2009.
- ▶ “TVM: An Automated End-to-End Optimizing Compiler for Deep Learning”, Chen et al., 2018.

Runtimes

- ▶ ONNX Runtime: <https://onnxruntime.ai/docs>
- ▶ LiteRT: <https://ai.google.dev/edge/litert>
- ▶ NCNN: <https://github.com/Tencent/ncnn>
- ▶ ExecuTorch: <https://executorch.ai>

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. github.com/Mjrovai/EdgeML_Made_Ease_ebook and github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.